Analysis and Design of Algorithms

CS3230

Approximation Algorithms

- Knapsack problem
- Travelling Salesman
- Set Cover
- FPTAS for Knapsack

A/P Lee Wee Sun

Approximation Algorithm

NP-Complete problems are intractable (unless P = NP).

- Can try to solve small instances optimally (Brute force through enumeration or cleverer methods such as branch and bound).

- Check if your problem belongs to a special case that can be solved efficiently
  - **Example:** Knapsack problem with small capacity -- can be solved with a pseudo-polynomial time dynamic programming algorithm
- Try to find *near-optimal* solutions in polynomial time.

Approximation Ratio

Let C^* be the optimal cost and let C be the cost of approximation algorithm A.

An approximation algorithm A has an *approximation ratio* of $\rho(n)$ if:

- $C/C^* \leq \rho(n)$ (for minimization) or $C/C^* \geq \rho(n)$ (for maximization)

Algorithm A is a $\rho(n)$ approximation algorithm (or ρ-approximation if independent of n)

Knapsack Problem

KNAPSACK

Input:

$(w_1, v_1), (w_2, v_2), \ldots, (w_n, v_n)$, and W

Output: A subset $S \subseteq \{1, 2, \ldots, n\}$ that maximizes

$\Sigma_{i \in S} v_i$ such that $\Sigma_{i \in S} w_i \leq W$

The decision version of this problem is NP-complete.

A Greedy Algorithm

Possible greedy criterion:

- Weight ?
- Value ?

Value/Weight ratio seems more sensible: greatest value for every unit of weight used.

- Assume $v_1/w_1, v_2/w_2, \ldots, v_n/w_n$ are in sorted order with the largest ratio first.
- Add items in order while $\Sigma_{i \in S} w_i \leq W$

Bad news: Approximation ratio of this greedy algorithm can be bad. Consider, two items with total weight constraint W:

- $(w_1,v_1) = (1,2)$
- $(w_2,v_2) = (W, W)$

Greedy algorithm selects (w_1,v_1) and is unable to further fit (w_2,v_2). Optimal algorithm will select (w_2,v_2). Hence, approx ratio is $2/W$, arbitrarily bad as W increases.

Consider adding another step of comparing

- the greedy solution
- the item with the *largest value (*and weight less than *W)*

and selecting the better of the two choices.

Call this the **modified greedy knapsack algorithm**. Surprisingly, approx ratio is 0.5!

To prove the approximation ratio, we first consider the **fractional knapsack problem**.

The fractional knapsack problem is the same as the knapsack problem except that we can choose a fraction $\alpha_i w_i$ of value $\alpha_i v_i$ for each item i, where α_i is an arbitrary fraction.

Greedy Fractional Knapsack Algorithm

As in the earlier algorithm, we

- Assume $v_1/w_1, v_2/w_2, \ldots, v_n/w_n$ are in sorted order with the largest ratio first.
- Add item in order while $\Sigma_{i \in S} w_i \leq W$

However we add another step

- For the first item that cannot fit, fit in the *largest fraction possible* so that the sum of weights is W.

The greedy algorithm is optimal.

Optimal substructure: Let (w_i, v_i) be an item which is in the optimal solution with fraction α_i. The optimal value is equal to $\alpha_i v_i$ plus the optimal value of the same problem with constraint $W - \alpha_i w_i$ and the $n-1$ original items plus $(1-\alpha_i) w_i$ of item i.

Proof: (*Cut and Paste*). Assume not. Then we can replace the solution of the subproblem (problem with constraint W-$\alpha_i w_i$ and n-1 original items and $(1-\alpha_i)w_i$ of item i) with the optimal solution and improve the on the optimal solution. Contradiction. □

Greedy choice: There exists an optimal solution where the item (w_i, v_i) with the largest ratio of v_i/w_i belongs to the optimal solution with fraction $\alpha_i = \min\{1, (W/w_i)\}$.

Proof: (*Cut and Paste*). Consider an optimal solution sorted largest ratio first. Replace the first $\alpha_i w_i$ amount of weight.

Item i has the largest ratio.

If the first $\alpha_i w_i$ amount of weight has lower value, we can replace it with $\alpha_i w_i$ of item i and we have a contradiction.

If the value is the same, replacing it does not change the value of the solution.

Hence there exists an optimal solution with $\alpha_i w_i$ of item i. $\square$

The *greedy choice property* together with the *optimal substructure property* implies that the greedy fractional knapsack algorithm is optimal.

Approximation ratio of greedy algorithms

We will refer to the original knapsack problem as the **0-1 knapsack problem** to distinguish it from the fractional knapsack problem.

Theorem: The approximation ratio for the modified greedy 0-1 knapsack algorithm is at least 0.5.

Proof: We first get an equivalent problem by removing each item i with $w_i > W$.

Suppose the greedy first part of the algorithm selects k items of total value V_1.

Let the value of the most valuable item be V_2. This is greater than v_{k+1}, the value of the next item in the list.

Let the final value of the modified greedy 0-1 knapsack algorithm be $V' = \max\{V_1, V_2\}$, and V_f be the value of the greedy fractional knapsack algorithm.

We have

$V' \geq V_1$

$V' \geq V_2 \geq v_{k+1}$

Adding the two eqns:

$$2V' \geq V_1 + v_{k+1}$$
$$\geq V_f$$
$$\geq V_{opt}$$

So $V'/V_{opt} \geq 1/2$. $\square$

V_1: first k items
V_2: most valuable item
V': modified greedy 0-1
V_f: greedy fractional
V_{opt}: optimal

Recap

Poly-time approx algorithms with guaranteed approx ratio

Simple approx algorithm for knapsack problem with approx ratio of 0.5

- Greedy algorithm with final step of comparing with highest value item

Analysis and Design of Algorithms

CS3230

Approximation Algorithms

- Knapsack problem
- **Travelling Salesman**
- Set Cover
- FPTAS for Knapsack

A/P Lee Wee Sun

Travelling Salesman Problem (TSP)

Input:

- A complete undirected graph $G = (V, V \times V)$.
- A nonnegative cost function $c: V \times V \rightarrow Z$.

Output: A tour that visits all vertices in V with the minimum cost.

The decision version of this problem is NP-complete.

Triangle Inequality

Triangle Inequality:

$c(u, w) \leq c(u, v) + c(v, w)$

for all vertices $u, v, w \in V$.

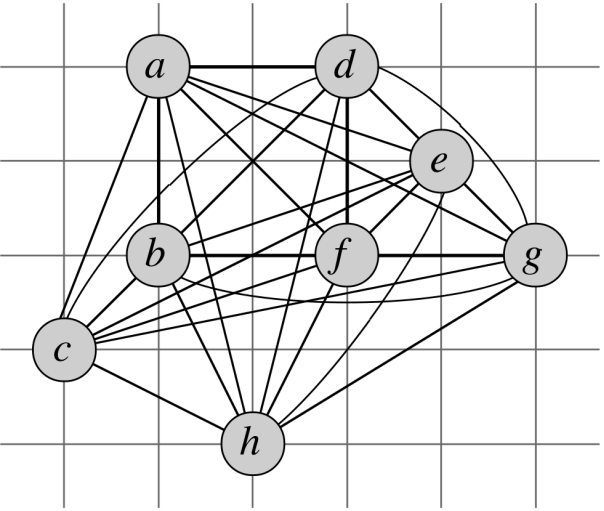
This assumption is valid for many of TSP applications (e.g. visiting all cities in a country).

TSP with this additional assumption (METRIC-TSP) is still NP-Complete.

Approximation Algorithm

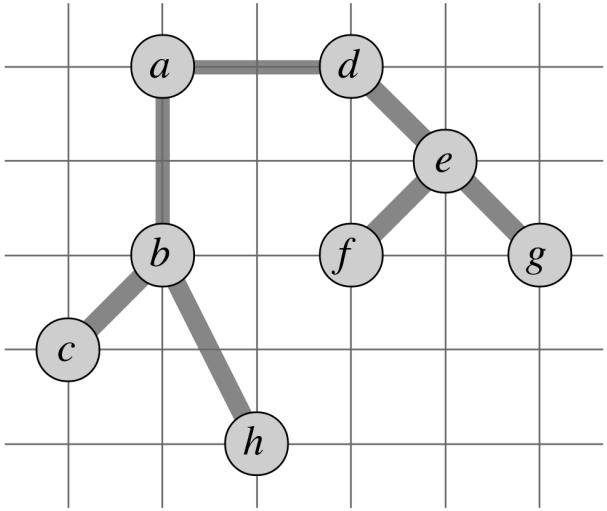
APPROX-TSP-TOUR(G)

1. Compute a Minimum Spanning Tree T for G
2. Let L be the list of vertices visited in preorder tree walk of T
3. Return Hamiltonian cycle H that visits the vertices in the order of L, skipping repeated vertices



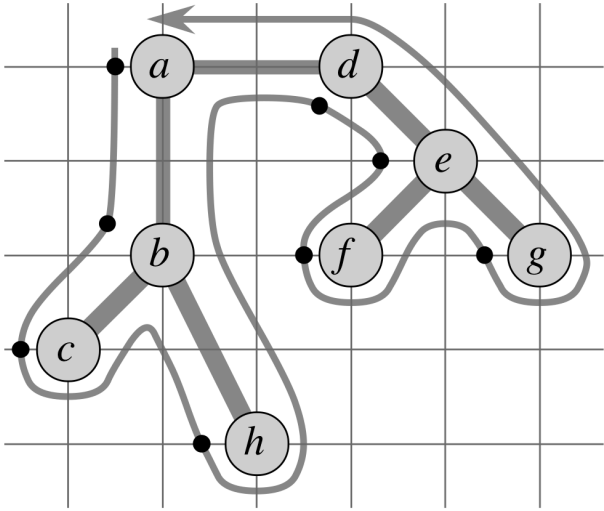
(a)

Graph



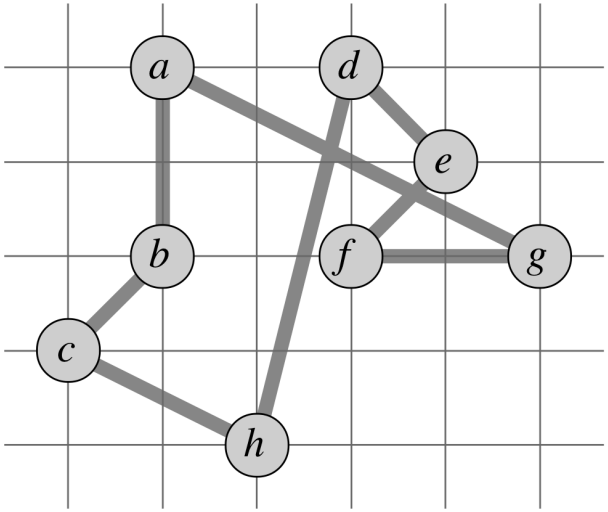
(b)

Spanning Tree

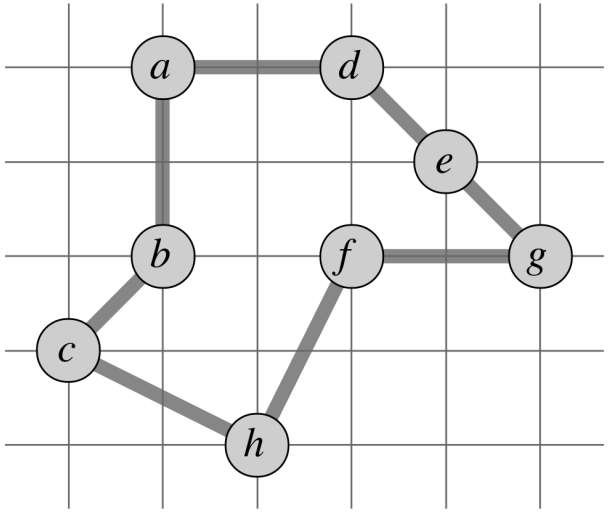


(c)

Pre-order walk



(d)



(e)

Resulting tour

Optimal tour

Polynomial Runtime

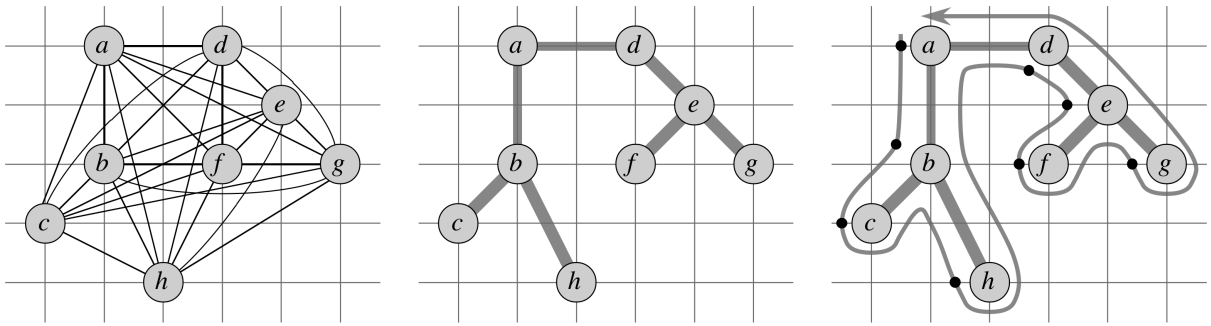
APPROX-TSP-TOUR(G)

1. Compute a Minimum Spanning Tree T for G. **Runtime:** $\Theta(V^2)$
2. Let L be the list of vertices visited in preorder tree walk of T **Runtime:** $\Theta(V)$
3. Return Hamiltonian cycle H that visits the vertices in the order of L, skipping repeated vertices

2-Approx Algorithm

Let C^* be the cost of the optimal tour and C_T be the sum of cost of edges in the MST T.

Then $C^* \geq C_T$ (Best tree *vs* best tour)



Let C_H be the sum of cost of the Hamiltonian cycle H

- Let H' be the tour that pass through each tree edge twice in the traversal instead of skipping repeated vertices, $C_{H'} \leq 2C_T$
- Repeated vertices skipped in H: by triangle inequality, $C_H \leq C_{H'} \leq 2C_T$

Hence, $C_H \leq 2C_T \leq 2C^*$ giving a 2-approx.

General TSP

Theorem: There is no polynomial time constant-approximation algorithm that can solve the general form of TSP unless P = NP.

Proof:

Idea: Show that HAM-CYCLE $\leq_P \rho$-APPROX-TSP

HAM-CYCLE $\leq_P$ ρ-APPROX-TSP

Reduction: Given $G = (V, E)$ where $n = |V|$, construct $G' = (V, V \times V)$.

For every $u, v \in V$

$$c(u,v) = \begin{cases} 1 & \text{if } (u,v) \in E \\ \rho n + 1 & \text{otherwise} \end{cases}$$

This reduction is clearly polynomial time: $O(V + E)$.

Lemma: The optimal solution for G' is $\leq n$ if G has a Hamiltonian cycle and $> \rho n$ if G does not have a Hamiltonian cycle.

If G contains a Hamiltonian cycle $(v_1, v_2, v_3, \ldots, v_{n-1}, v_n, v_1)$, then, the cost of the same tour in G' is $c(v_1, v_2) + c(v_2, v_3) + \ldots + c(v_{n-1}, v_n) + c(v_n, v_1) = n$.

If G does not contain a Hamiltonian cycle, any tour in G' contains at least one edge not in E, which costs a heavy penalty of $\rho n+1$.

Hence, the cost of the tour is at least $\rho n+1 > \rho n$. $\square$

No Approximation

Assume we have a ρ-approximation algorithm which we use to solve G'.

From the lemma, we know that the algorithm will output a solution with $\leq \rho n$ if G has a Hamiltonian cycle and $> \rho n$ if G does not have a Hamiltonian cycle.

Therefore the algorithm can be used to solve Hamiltonian cycle efficiently, contradicting $P \neq NP$.

Proving non-approximability

To show that Y cannot be approximated well, we reduce an NP-complete (decision) problem X to (decision version of) APPROX-Y

- "yes" instances of X correspond to instances of Y with value at most k
- "no" instances of X correspond to instances of Y with value greater than ρk

Unless P = NP, there is no polynomial time ρ-approximation algorithm for Y

Recap

2-approx algorithm for METRIC-TSP

- Construct min spanning tree (relaxation)
- Pre-order traversal
- Skip repeated vertices to get tour

No poly time const-approx algorithm for general TSP unless P=NP

Analysis and Design of Algorithms

CS3230

Approximation Algorithms

- Knapsack problem
- Travelling Salesman
- **Set Cover**
- FPTAS for Knapsack

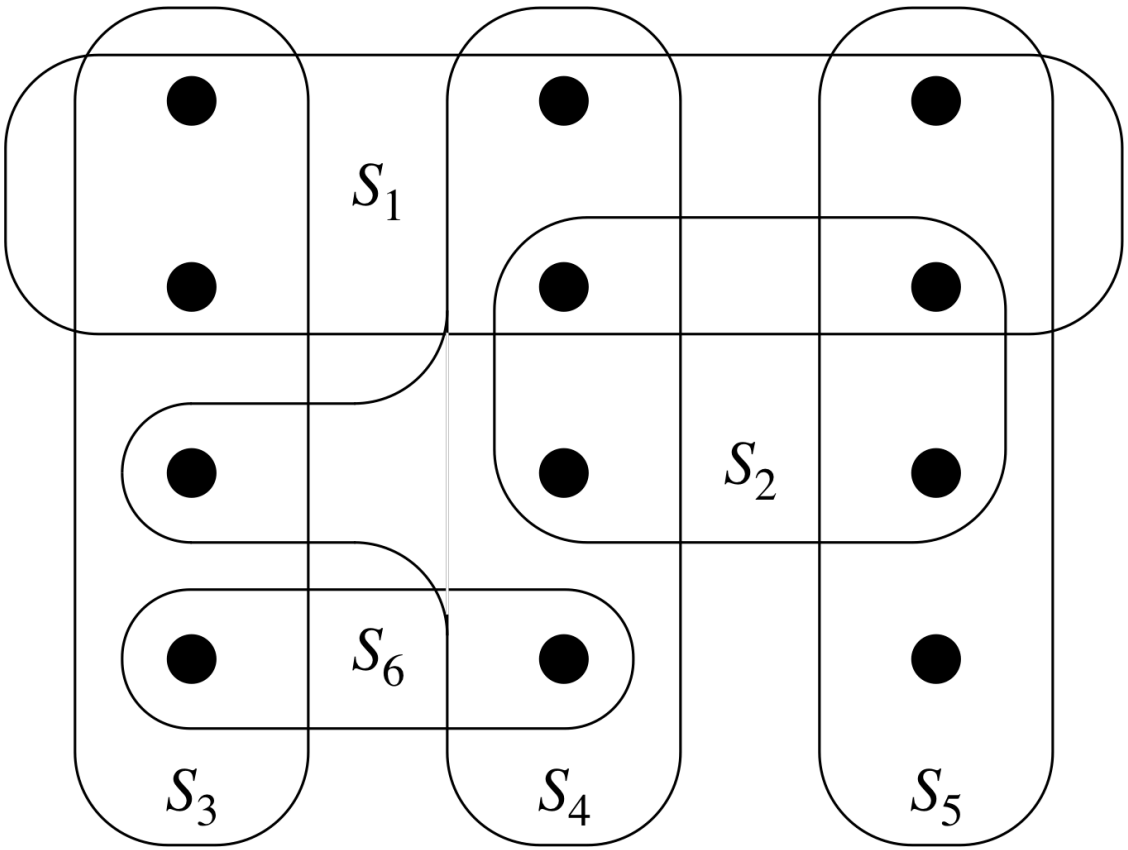
A/P Lee Wee Sun

Set Cover

Input: (X, F) where X is a finite set and F is a family of subsets of X such that every element of X belongs to at least one subset in F,
i.e. $X = \cup_{S \in F} S$

Output: Find the minimum size set cover C

Definition: $C \subseteq F$ is called a set cover if $X = \cup_{S \in C} S$



Minimum cover $C_{opt} = \{S_3, S_4, S_5\}$

Greedy Set Cover

GREEDY-SET-COVER(X,F)

$U \leftarrow X$, $C \leftarrow \Phi$

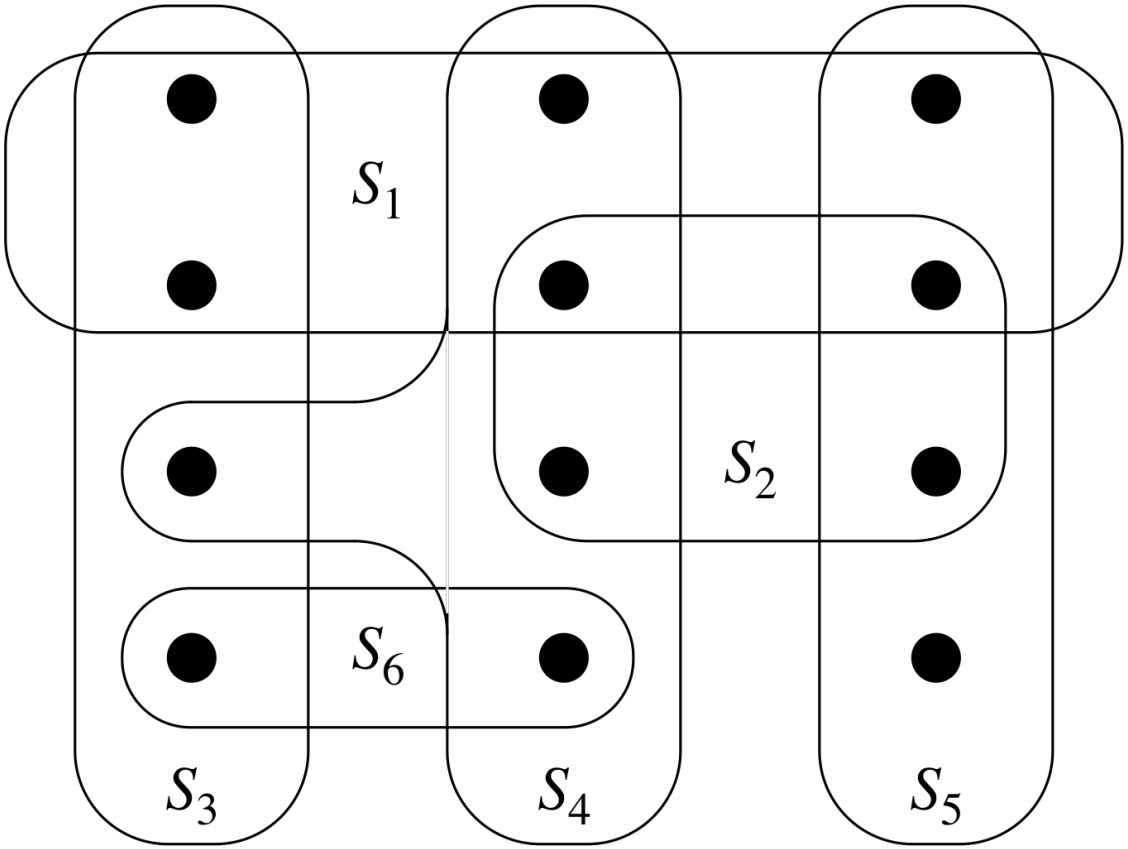
while $U \neq \Phi$

  do select an $S \in F$ that maximizes $|S \cap U|$

    $U \leftarrow U - S$

    $C \leftarrow C \cup \{S\}$

Return C



GREEDY-SET-COVER selects S_1, S_4, S_5, S_3 in sequence while minimum cover

$C_{opt} = \{S_3, S_4, S_5\}$

Analysis

Theorem: GREEDY-SET-COVER is a polynomial-time $\lceil \ln |X| \rceil$ -approximation algorithm.

Proof: Let opt denote the size of the minimum cover. Consider the i-th greedy step.
Claim: There must be a set that covers at least $|U_{i-1}|/opt$ elements, where U_{i-1} is the number of remaining elements before the i-th step.

First, note that the same *opt* sets in the optimal cover also covers all elements of U_{i-1}.

Hence, there exists a set that covers at least $|U_{i-1}|/opt$ elements. Otherwise cannot cover all elements using all *opt* sets – contradiction. □

The number of *uncovered* elements after the i-th greedy step is no more than $|U_{i-1}|-|U_{i-1}|/opt = (1-1/opt)|U_{i-1}|$.

Applying the argument repeatedly for k rounds, we get

$|U_k| \leq (1-1/opt)^k |X| < e^{-k/opt} |X|$

where we used $(1-x) \leq e^{-x}$.

Hence if $e^{-k/opt} |X| \leq 1$, we would have $|U_k| < 1$. Taking log on both sides, we get $-k/opt \leq \ln 1/|X|$, so $k \geq opt \ln |X|$.

Setting $k = opt \lceil \ln |X| \rceil$ is sufficient. $\square$

Recap

Greedy approx algorithm for set cover

- Select set that covers the largest number of remaining elements (greedy)

Log $|X|$ approximation ratio

Analysis and Design of Algorithms

CS3230

Approximation Algorithms

- Knapsack problem
- Travelling Salesman
- Set Cover
- **FPTAS for Knapsack**

A/P Lee Wee Sun

Polynomial time approximation scheme

In the previous examples for knapsack and metric TSP, the approximation ratio is fixed at 0.5 and 2 respectively. Adjust approx ratio?

An approximation algorithm whose approximation ratio is $1+\varepsilon$ (or $1-\varepsilon$ for maximization) for any $\varepsilon > 0$, is called a **polynomial-time approximation scheme** (**PTAS**).

Fully polynomial time approximation scheme

As ε decreases, the runtime for a PTAS increases, potentially exponential in $1/\varepsilon$ (e.g. $O(n^{1/\varepsilon})$).

If the running time of the PTAS is polynomial in both $1/\varepsilon$ and n (e.g $O((1/\varepsilon)^2 n^3)$), it is a **fully polynomial-time approximation scheme** (**FPTAS**).

FPTAS for Knapsack

$(1-\varepsilon)$-approximation for any $\varepsilon > 0$

Runtime increases polynomially with $1/\varepsilon$.

User selects ε to trade off accuracy with runtime.

For knapsack, we will use rounding followed by a pseudo-polynomial time dynamic programming algorithm to obtain a FPTAS.

Approach

Recall the pseudo-polynomial time algorithm for knapsack covered earlier.

If w_i and W are integers, runtime $O(nW)$.

We are interested in approximating value, not weight. So, we use a different dynamic programming algorithm where v_i are integers and runtime is $O(n^2 v_{max})$ where $v_{max} = \max v_i$.

Solution is always feasible.

Step 1: Set $v'_i = \lfloor v_i/q \rfloor$ for each i, where $\lfloor x \rfloor$ denotes the floor of x and q is a parameter to be specified later.

Step 2: Use DP to solve knapsack with $(w_1,v'_1),\ldots,(w_n,v'_n)$ and capacity W with running time $O(n^2 v'_{max})$.

Dynamic Programming

Let $i = 0, 1, \ldots, n$ be item index,
$y = 0, 1, \ldots, nv_{max}$, be possible values.

$S[i,y]$ be smallest total weight required to achieve value at least y using the first i items.

$$S[i,y] = \min \begin{cases} S[i-1,y] \\ w_i + S[i-1, \max\{0, y - v_i\}] \end{cases}$$

KNAPSACK(v, w, W)

$S[0,0] = 0$

for $y = 1$ **to** nv_{max} ▷ Initialization

$S[0, y] = \infty$

▷ Recursive cases

…

return largest y such that $S[i, y] \leq W$

$$S[i,y] = \min \begin{cases} S[i-1,y] \\ w_i + S[i-1, \max\{0, y - v_i\}] \end{cases}$$

▷ Recursive cases

Runtime: $\Theta(n^2 v_{max})$

for $i \leftarrow 1$ **to** n

  for $y \leftarrow 0$ **to** nv_{max}

    $S[i,y] \leftarrow w_i + S[i\text{-}1, \max\{0, y\text{-}v_i\}]$

    if $S[i,y] > S[i\text{-}1,y]$ **then**

      $S[i,y] \leftarrow S[i\text{-}1,y]$

Analysis

Recall that we set $v'_i = \lfloor v_i/q \rfloor$ for each i.

What value of q do we need to use to ensure a $(1-\varepsilon)$ approximation ratio?

What is the running time for that value of q?

Because we set $v'_i = \lfloor v_i/q \rfloor$, we have $qv'_i \in [v_i\text{-}q, v_i]$ for each i implying

(1) $v_i \geq qv'_i$

(2) $qv'_i \geq v_i - q$

Let S^* be the optimal solution to the original problem and S be the optimal solution to the rounded problem. We have

(3) $\sum_{i \in S} v'_i \geq \sum_{i \in S^*} v'_i$

(1) $v_i \geq qv'_i$
(2) $qv'_i \geq v_i - q$
(3) $\sum_{i \in S} v'_i \geq \sum_{i \in S^*} v'_i$

From the three inequalities, we get

$$\sum_{i \in S} v_i \geq q \sum_{i \in S} v'_i \qquad \text{from (1)}$$

$$\geq q \sum_{i \in S^*} v'_i \qquad \text{from (3)}$$

$$\geq \sum_{i \in S^*} (v_i - q) \qquad \text{from (2)}$$

So we have

$$\sum_{i \in S} v_i \geq \left(\sum_{i \in S^*} v_i\right) - nq$$

So far we have $\sum_{i \in S} v_i \geq (\sum_{i \in S^*} v_i) - nq$

and we want $\sum_{i \in S} v_i \geq (1 - \varepsilon) \sum_{i \in S^*} v_i$

Suffices if we find q such that $\varepsilon \sum_{i \in S^*} v_i \geq nq$

Then

$$\sum_{i \in S} v_i \geq (\sum_{i \in S^*} v_i) - nq \geq \sum_{i \in S^*} v_i - \varepsilon \sum_{i \in S^*} v_i$$

We have $\sum_{i \in S^*} v_i \geq v_{max}$ as $\sum_{i \in S^*} v_i$ is the optimal solution. So $nq = \varepsilon v_{max}$ is sufficient.

Runtime

Recall running time is $O(n^2 v'_{max})$ and

$v'_i \leq v_i/q$

$\leq v_{max}/q$

Substituting $q = \varepsilon v_{max}/n$

we have $v'_i \leq n/\varepsilon$, so total running time is

$O(n^3/\varepsilon)$.

Recap

Fully polynomial time approximation scheme (FPTAS) for knapsack

- Rounding and pseudo-polynomial time dynamic programming
- $O(n^3/\varepsilon)$ for $(1-\varepsilon)$ approximation ratio

Recap for approx algorithms

Constant approx ratio algorithms for knapsack and metric TSP, *logarithmic approx ratio* for set cover, *FPTAS* for knapsack

Some common techniques: *greedy, relaxation, dynamic programming*